

Web-APIs: REST und SOAP

Dieses Dokument wurde mit Unterstützung von ChatGPT erstellt.

REST und SOAP sind zwei unterschiedliche Architekturstile für den Entwurf von Web-APIs. Hier ist eine Übersicht über beide:

1 REST (Representational State Transfer)

- **Architekturprinzipien:**

- REST basiert auf einer Reihe von Prinzipien, die die Kommunikation zwischen Client und Server erleichtern. Es nutzt HTTP-Methoden (GET, POST, PUT, DELETE) zur Durchführung von Operationen.
- REST ist zustandslos, was bedeutet, dass jeder API-Aufruf alle Informationen enthält, die der Server benötigt, um die Anfrage zu verstehen und zu verarbeiten.

- **Datenformat:**

- REST ist flexibel hinsichtlich der Datenformate, die es verwenden kann (z.B. JSON, XML, HTML, Text). JSON ist jedoch das am häufigsten verwendete Format aufgrund seiner Einfachheit und Lesbarkeit.

- **Vorteile:**

- Einfach und leicht zu implementieren.
- Gut geeignet für öffentliche APIs und Webanwendungen.
- Skalierbar und effizient, da es HTTP-Standards nutzt.

- **Nachteile:**

- Kann bei komplexen Transaktionen weniger geeignet sein, da es keinen eingebauten Standard für Sicherheit oder Transaktionen bietet.

- **Funktionsweise:**

- **Grundprinzip:** RESTful APIs nutzen HTTP-Methoden, um mit Ressourcen zu interagieren. Ressourcen sind identifizierbar durch URLs (Uniform Resource Locators).
- **HTTP-Methoden:**
 - **GET:** Ruft eine Ressource ab.
 - **POST:** Erstellt eine neue Ressource.
 - **PUT:** Aktualisiert eine vorhandene Ressource.
 - **DELETE:** Löscht eine Ressource.
- **HTTP-Statuscodes:** Diese Codes werden verwendet, um den Status der HTTP-Anfrage zu kommunizieren.
 - **200 OK:** Die Anfrage war erfolgreich.
 - **201 Created:** Eine neue Ressource wurde erfolgreich erstellt.
 - **400 Bad Request:** Die Anfrage war fehlerhaft.

- **404 Not Found:** Die angeforderte Ressource wurde nicht gefunden.
 - **500 Internal Server Error:** Ein Serverfehler ist aufgetreten.
 - **Datenformat:** REST kann verschiedene Datenformate verwenden, wobei JSON am häufigsten ist. Eine typische REST-Antwort könnte so aussehen:

```
{
  "id": 1,
  "name": "Beispiel",
  "status": "active"
}
```
 - **Zustandslosigkeit:** Jede Anfrage von einem Client an den Server muss alle Informationen enthalten, die erforderlich sind, um die Anfrage zu verstehen und zu verarbeiten.
- **Beispiel:**
 - Abrufen von Informationen über einen Benutzer**
 - HTTP GET Request**

Stellen Sie sich vor, wir haben eine API, die Informationen über Benutzer bereitstellt. Um Informationen über einen Benutzer mit der ID 123 abzurufen, könnten wir einen HTTP GET Request an folgende URL senden:

```
GET /api/users/123 HTTP/1.1
Host: example.com
Accept: application/json
```
 - GET:** Die HTTP-Methode, die verwendet wird, um Daten vom Server abzurufen.
 - /api/users/123:** Der Endpunkt, der die Ressource (Benutzer mit ID 123) spezifiziert.
 - Host:** Der Server, an den die Anfrage gesendet wird.
 - Accept:** Der Header, der angibt, dass die Antwort im JSON-Format erwartet wird.
 - HTTP Response**

Der Server antwortet mit den Benutzerinformationen im JSON-Format:

```
HTTP/1.1 200 OK
Content-Type: application/json
{
  "id": 123,
  "name": "John Doe",
  "email": "john.doe@example.com",
  "status": "active"
}
```
 - HTTP/1.1 200 OK:** Der Statuscode 200 zeigt, dass die Anfrage erfolgreich war.
 - Content-Type:** Der Header, der das Format der Antwort angibt (in diesem Fall JSON).
 - JSON-Body:** Enthält die Daten der angeforderten Ressource, in diesem Fall Informationen über den Benutzer.

2 SOAP (Simple Object Access Protocol)

- **Architekturprinzipien:**

- SOAP ist ein Protokoll, das auf XML basiert und es ermöglicht, Nachrichten zwischen Client und Server zu übertragen. SOAP verwendet oft HTTP oder SMTP als Transportprotokoll.
- SOAP ist detaillierter und standardisierter, mit eingebauten Spezifikationen für Sicherheit (WS-Security) und Transaktionen.

- **Datenformat:**

- SOAP verwendet ausschließlich XML für Nachrichten. Dies macht es komplexer und schwerer zu lesen im Vergleich zu REST.

- **Vorteile:**

- Bietet eingebaute Sicherheits- und Transaktionsstandards, was es für Unternehmensanwendungen geeignet macht.
- Plattform- und sprachunabhängig.

- **Nachteile:**

- Komplexer und ressourcenintensiver als REST.
- Schwerer zu implementieren und zu warten.

- **Funktionsweise:**

- **Grundprinzip:** SOAP ist ein Protokoll, das XML-Nachrichten überträgt. Es ist unabhängig vom Transportprotokoll, wird aber häufig mit HTTP verwendet.
- **Nachrichtenstruktur:** Eine SOAP-Nachricht besteht aus einem Envelope, der den Header und den Body enthält.
 - **Envelope:** Der Container für die Nachricht.
 - **Header:** Optional, enthält Informationen wie Authentifizierung.
 - **Body:** Enthält die eigentliche Nachricht oder den Aufruf.
- **WSDL (Web Services Description Language):** SOAP-Dienste werden oft durch eine WSDL-Datei beschrieben, die die verfügbaren Methoden und deren Parameter definiert.

- **Beispiel einer SOAP-Nachricht:**

```
<soapenv:Envelope
xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/"
xmlns:ex="http://example.com/">
  <soapenv:Header/>
  <soapenv:Body>
    <ex:GetExample>
      <ex:Id>1</ex:Id>
    </ex:GetExample>
  </soapenv:Body>
</soapenv:Envelope>
```

- **Erweiterungen:** SOAP unterstützt Erweiterungen wie WS-Security für Sicherheit und WS-ReliableMessaging für zuverlässige Nachrichtenübertragung.

- **Beispiel:**

SOAP Request

SOAP verwendet XML für die Nachrichtenübertragung. Angenommen, wir möchten Informationen über einen Benutzer mit der ID 123 abrufen, könnte die SOAP-Anfrage wie folgt aussehen:

```
POST /UserService HTTP/1.1
Host: example.com
Content-Type: text/xml; charset=utf-8
SOAPAction: "http://example.com/GetUser"
```

```
<soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/" xmlns:ex="http://example.com/">
  <soapenv:Header/>
  <soapenv:Body>
    <ex:GetUserRequest>
      <ex:UserId>123</ex:UserId>
    </ex:GetUserRequest>
  </soapenv:Body>
</soapenv:Envelope>
```

- **POST:** Die HTTP-Methode, die verwendet wird, um die SOAP-Nachricht zu senden.
- **/UserService:** Der Endpunkt, der den Webservice spezifiziert.
- **Content-Type:** Gibt an, dass der Inhalt eine SOAP-Nachricht im XML-Format ist.
- **SOAPAction:** Ein HTTP-Header, der die beabsichtigte Aktion beschreibt.
- **SOAP-Envelope:** Der Container für die gesamte Nachricht, bestehend aus Header und Body.
- **SOAP-Body:** Enthält die eigentliche Anfrage, in diesem Fall GetUserRequest mit der UserId.

SOAP Response

Der Server antwortet mit einer SOAP-Nachricht, die die Benutzerinformationen enthält:

```
HTTP/1.1 200 OK
Content-Type: text/xml; charset=utf-8
```

```
<soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/" xmlns:ex="http://example.com/">
  <soapenv:Header/>
  <soapenv:Body>
    <ex:GetUserResponse>
      <ex:User>
        <ex:Id>123</ex:Id>
        <ex:Name>John Doe</ex:Name>
      </ex:User>
    </ex:GetUserResponse>
  </soapenv:Body>
</soapenv:Envelope>
```

```
<ex:Email>john.doe@example.com</ex:Email>
<ex:Status>active</ex:Status>
</ex:User>
</ex:GetUserResponse>
</soapenv:Body>
</soapenv:Envelope>
```

HTTP/1.1 200 OK: Der Statuscode zeigt an, dass die Anfrage erfolgreich war.

SOAP-Envelope: Enthält den Header und den Body der Antwort.

SOAP-Body: Enthält die Antwortdaten, in diesem Fall GetUserResponse mit den Benutzerinformationen.

3 Zusammenfassung

- **REST** ist ideal für einfache, skalierbare Webservices, die auf HTTP basieren und JSON als bevorzugtes Datenformat verwenden.
- **SOAP** eignet sich besser für Anwendungen, die erweiterte Sicherheits- und Transaktionsanforderungen haben und XML-basierte Kommunikation erfordern.

REST ist einfacher und nutzt die bestehenden Fähigkeiten des HTTP-Protokolls, während SOAP ein komplexeres Protokoll ist, das zusätzliche Spezifikationen für Sicherheit und Transaktionen bietet.

Weiterführende Links zum Thema:

<https://stoplight.io/api-types/soap-api>

<https://www.uptrends.de/was-ist/rest-api>

https://www.w3schools.com/tags/ref_httpmessages.asp